Defiende el servidor: UNSW-NB15

MLOps · 2026-II · María José Cortés Romero (Grupo A) · 29 de octubre de 2026 · Código: github.com/Mari4Z/Tallercito

Resumen. Clasificador binario normal/ataque (ataque = clase positiva). Se comparan cuatro configuraciones de tres familias; v1 es la regresión logística (umbral 0,50) y v2 el HistGradientBoosting con umbral 0,65, elegido solo con validación. En validación v2 reduce las falsas alarmas (FPR 0.133 → 0.049) a cambio de perder más ataques (recall 0.975 → 0.953). En el test final **empeora**: FPR 0.175 y macro-F1 0.900. Todo es recuperable y verificable con los identificadores de la sección 6.

1. Datos y limpieza

Se usan las particiones publicadas de UNSW-NB15 (copia espejo del anexo; los SHA-256 coinciden): entrenamiento 175.341 filas y test 82.332, 45 columnas, **sin valores faltantes**. Entradas: 42 variables (3 categóricas: `proto`, `service`, `state`); se excluyen `label`, `attack_cat` (revela la respuesta) e `id`. El EDA mostró que el `id` delata la clase (el archivo está ordenado por tipo de tráfico: el primer 25 % del entrenamiento es 100 % normal y el último 30 % es 100 % ataque).

Hallazgo (auditoria.json)	Entrenamiento	Test
Filas con entradas idénticas a otra (solo variables predictoras)	74,301 (42.38 %)	28,386 (34.48 %)
Grupos de entradas iguales con etiquetas contradictorias (filas)	229 (940)	6 (14)
Test con entradas idénticas a train/validación	8,541 filas (10.37 %)	

Política. No se elimina ninguna fila (la distribución se conserva y el test se evalúa completo). Los grupos de entradas idénticas, incluidos los contradictorios, se asignan enteros a un solo lado con `StratifiedGroupKFold` (semilla 42, 5 pliegues, uno como validación): train 140,272 filas (68.06 % ataque) y validación 35,069 (68.06 %), con **0 grupos compartidos**. Codificación y escalado se ajustan solo con train, dentro del pipeline que se guarda completo. La partición es idéntica para todos los experimentos y se reproduce bit a bit (DVC: datos originales y preparados, dos versiones).

2. Experimentos (MLflow, validación, umbral 0,5, mismas particiones)

Las hipótesis se escribieron y commitearon antes de entrenar (`configs/experiments.yaml`). Métricas con ataque como clase positiva; macro-F1 = promedio del F1 de ataque y del F1 de normal; si un denominador es 0 el cociente vale 0,0 y se marca `zero_division` (no ocurrió).

ID	Cambio	Hipótesis	Recall	Prec.	FPR	M-F1	Resultado
E0	Piso (DummyClassifier, prior)	Predice siempre ataque: recall 1, FPR 1, macro-F1 muy bajo.	1.000	0.681	1.000	0.405	Confirmada
E1	Referencia: regresión logística + log1p + escalado	El modelo lineal detecta casi todo pero alarma con FPR > 0,10.	0.975	0.940	0.133	0.930	Confirmada
E2	Cambia de familia: HistGradientBoosting	El boosting tiene el mejor macro-F1 y menor FPR que la logística.	0.975	0.962	0.083	0.950	Confirmada
E3	Cambia de familia: random forest (100 árboles, min_samples_leaf=5)	El bosque queda a menos de un punto de macro-F1 del boosting.	0.981	0.952	0.106	0.945	Confirmada

Una primera ronda con otros modelos (regresión logística balanceada, bosques de 200 árboles con y sin `sttl`) se descartó al cambiar el conjunto de modelos y no forma parte de la entrega; sus hipótesis se habían contradicho en tres de cinco casos (p. ej. `class_weight="balanced"` bajó el recall porque aquí la clase minoritaria es la normal). Las hipótesis de E1 y E3 están por ello informadas por esa ronda.

Lectura. El piso muestra que la accuracy engaña (0,681 sin aprender nada). Los modelos de árboles superan al lineal sobre todo en falsas alarmas; el boosting es el mejor (macro-F1 0.950) y su archivo pesa 2,06 MB frente a 28,8 MB del bosque, así que elegir por evidencia llevó al boosting y no al algoritmo más conocido.

3. Cambios entre v1 y v2 (solo validación)

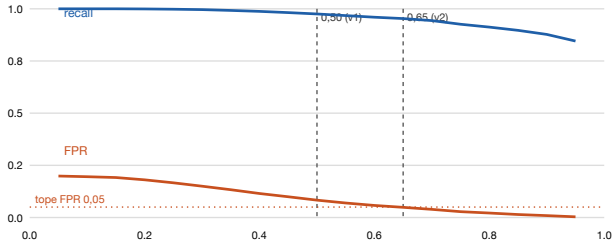
Métrica	v1 · logística · 0,50	v2 · boosting · 0,65
Recall	0.975	0.953
Precision	0.940	0.977
FPR	0.133	0.049
Macro-F1	0.930	0.946
Ataques perdidos (FN)	590	1117
Falsas alarmas (FP)	1485	545

v2 cambia **dos cosas**: el modelo y el umbral. Solo el modelo (boosting con umbral 0,5): recall 0.975 (igual que v1) y FPR 0.083. El umbral 0,65 baja la FPR a 0.049.

Mejora: –940 falsas alarmas (–63 %), más precision y macro-F1. **Empeora:** recall –2,2 puntos (527 ataques más sin detectar). **Se acepta** perder más ataques a cambio de no saturar al equipo de seguridad.

4. Elección del umbral (validación)

Regla fijada antes de mirar el test: **el umbral más bajo cuya FPR en validación no supere 0,05** (como máximo una falsa alarma por 20 conexiones normales), lo que maximiza el recall bajo ese presupuesto. Resultado: **0.65**. El umbral que maximiza el macro-F1 (0,52) casi coincide con el 0,5 por defecto y no cambiaría nada operativamente. La decisión es de criterio: quien priorice no perder ningún ataque elegiría un umbral menor. Se congeló en el commit 1667e8d y la selección completa en 6b50021, ambos anteriores a cualquier evaluación en test. Las métricas de validación son algo optimistas porque el umbral se ajustó sobre esos mismos datos.



Recall y FPR de v2 en validación según el umbral.

5. Test final (una sola evaluación, v2 congelada)

Se evaluó la revisión de Hugging Face `53384879be28…` (hash del modelo verificado) sobre las **82,332 filas** del archivo de test, sin ajustes. Las predicciones por fila y las métricas están en `evidencias/` y se recalcularon desde el CSV con resultado idéntico.

	Recall	Precision	FPR	Macro-F1	TP	FP	TN	FN
Validación	0.9532	0.9766	0.0487	0.9462	22,752	545	10,655	1,117
Test	0.9653	0.8713	0.1747	0.8998	43,759	6,464	30,536	1,573

El resultado empeora y se conserva. El recall sube (0.965), pero la FPR se multiplica por 3,6: en test se alarma al 17.5 % del tráfico normal, no al 5 %. Causa probable (no comprobada): la validación sale de la misma distribución del entrenamiento, y el test oficial tiene otra mezcla (55 % de ataques frente a 68 %). Con estas cifras, v2 no cumple en test el presupuesto de falsas alarmas que se pretendía.

6. Trazabilidad

	v1	v2
Código (Git)	tag `modelo-v1`, `846fecaf`	entrenado en `846fecaf`; selección en tag `modelo-v2` (`6b50021`)
Datos (DVC)	preparados `4458575700…dir` · originales `fe93051ae2…dir` · remoto público de solo lectura en Hugging Face	
Experimento (MLflow)	`5892811f7423…`	`09cf945f4cc1…`
Modelo (Hugging Face, SHA)	`308d2e786ae5…`	`53384879be28…`
SHA-256 del modelo	`76d6c306f707…`	`544c94bdd84c…`

Recuperación. En una carpeta y entorno nuevos, sin caché ni credenciales, se clonó el código, `dvc pull` recuperó los datos (4 huellas coinciden con las auditadas) y se descargaron v1 y v2 por SHA completo. Sobre 50 filas fijas de validación las predicciones coinciden con las originales (diferencia máxima $1{,}1 \cdot 10^{-16}$, tolerancia 10^{-9}; archivos de salida idénticos por hash). Una primera prueba falló porque DagsHub exige sesión para descargar y obligó a cambiar el almacenamiento a un dataset público de Hugging Face.

7. Limitaciones

- Benchmark histórico de laboratorio: no demuestra eficacia sobre una red actual. Solo clasificación binaria.
- `sttl` y `ct_state_ttl` separan casi solas las clases (p. ej. `sttl=31`: 39.455 filas, ninguna ataque) por cómo se montó el laboratorio. Quitarlas no baja el rendimiento porque otras variables llevan información equivalente.
- Ruido de etiquetas (229 grupos contradictorios), duplicados masivos y 10.37 % del test coincidente con train/validación pueden inflar los resultados.
- El umbral se ajustó sobre la validación y no se corrigió por el desplazamiento observado en test; no se probó otra regla porque el test no puede usarse para elegir.
- La fecha de descarga de los CSV no se registró (consta la fecha de modificación del archivo). Los datos se hospedan públicamente para la revisión, con atribución a Moustafa y Slay (2015, MilCIS, IEEE) y condiciones de uso académico de UNSW.

Uso de IA. Se usó Claude Code (Claude Sonnet 5.5) para escribir código, ejecutar experimentos y redactar documentación, con verificaciones descritas en `entrega.md`; las decisiones de diseño (modelos, regla de umbral, publicación de datos) son de la autora.